

Програмні засоби наукових обчислень

Лабораторна робота №4

Варіант 3

Студентки 2-го курсу
Спеціальності ПМ
Гирба М.С

<u>Завдання №1.....</u>	<u>3</u>
Завдання №2.....	3
Завдання №3.....	3
Завдання №4.....	4
Завдання №5.....	4
Завдання №6.....	4
Завдання №7.....	5
Завдання №8.....	5

Задание 1

Условие :

По данным из файла `vaccination_process_2021_regions.xlsx` построить столбцовую диаграмму количества людей, вакцинировавшихся вакциной AstraZeneca в Одесской области, относительно месяца. За количество вакцинированных в определённом месяце брать сумму вакцинированных по имеющимся дням данного месяца.

Код решения:

```
import pandas as pd
import matplotlib.pyplot as plt

# Загрузка данных из файла
vaccination_data =
pd.read_excel('vaccination_process_2021_regions.xlsx')

# Фильтрация данных для вакцины AstraZeneca в Одесской области
vaccination_data['Month'] = pd.to_datetime(vaccination_data['Дата
(период) данных'], dayfirst=True).dt.month
odessa_data_astra = vaccination_data[(vaccination_data['Назва
території'] == 'Одеська область') & (vaccination_data['AstraZeneca,
осіб'] > 0)].copy()

# Суммирование вакцинированных по месяцам
monthly_vaccinated_astra =
odessa_data_astra.groupby('Month')['AstraZeneca, осіб'].sum()

# Построение столбиковой диаграммы
plt.figure(figsize=(10, 6))
monthly_vaccinated_astra.plot(kind='bar')
plt.xlabel('Месяц')
plt.ylabel('Количество вакцинированных')
plt.title('Количество людей, вакцинированных вакциной AstraZeneca в
Одесской области по месяцам')
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```

Результат тестов:



Задание 2

Условие :

К данным из файла filter3.xlsx добавить столбец Subsidy (субсидия), который определяется следующим образом: субсидия выделяется тем, у кого нет долгов по квартплате и чей доход меньше 4500. Субсидия начисляется в размере 10% от квартплаты. Если субсидия не начисляется, значение Subsidy в данной строке должно равняться нулю.

Отобразить информацию о квартиросъемщиках:

- 1) получающих субсидию
- 2) зарабатывающих меньше 4500 и имеющих задолженность по квартплате
- 3) чья квартплата превышает 2000
- 4) имеющих зарплату от 3000 до 5000
- 5) получающих субсидию, чья квартплата превышает 1500
- 6) получающих субсидию с квартплатой свыше 1800, а также лиц, получающих зарплату менее 5000 и имеющих задолженность по квартплате

- 7) выбрать всех лиц с задолженностью по квартплате, чья квартплата не превышает 1600, а также лиц, получающих зарплату от 3000 до 5000, чья квартплата превышает 1800

Код решения:

```
# Задание 2: Добавить столбец Subsidy и отобразить информацию о
квартиросъемщиках по различным критериям
import pandas as pd
import numpy as np

# Загрузка данных из файла
filter_data = pd.read_excel('filter3.xlsx')

# Проверка наличия нужных столбцов
required_columns = ['Rent', 'Debts', 'AverageIncome']
missing_columns = [col for col in required_columns if col not in
filter_data.columns]
if missing_columns:
    print(f"Отсутствуют необходимые столбцы в файле filter3.xlsx:
{missing_columns}")
else:
    # Преобразование столбцов 'Debts' и 'AverageIncome' в числовой тип
данных для корректной работы с условиями
    filter_data['Debts'] = pd.to_numeric(filter_data['Debts'],
errors='coerce').fillna(0)
    filter_data['AverageIncome'] =
pd.to_numeric(filter_data['AverageIncome'], errors='coerce').fillna(0)

    # Добавление столбца Subsidy по описанным критериям
    filter_data['Subsidy'] = np.where((filter_data['Debts'] == 0) &
(filter_data['AverageIncome'] < 4500), filter_data['Rent'] * 0.1, 0)

    # 1. Квартиросъемщики, получающие субсидию
    subsidy_recipients = filter_data[filter_data['Subsidy'] > 0]
    print("Получающие субсидию:")
    print(subsidy_recipients)

    # 2. Зарабатывающие меньше 4500 и имеющие задолженность по
квартплате
```

```

    debt_low_income = filter_data[(filter_data['AverageIncome'] < 4500)
& (filter_data['Debts'] > 0)]
    print("Зарабатывающие меньше 4500 и имеющие задолженность:")
    print(debt_low_income)

# 3. Квартиросъемщики, чья квартплата превышает 2000
high_rent = filter_data[filter_data['Rent'] > 2000]
print("Квартиросъемщики с квартплатой больше 2000:")
print(high_rent)

# 4. Квартиросъемщики с зарплатой от 3000 до 5000
income_range = filter_data[(filter_data['AverageIncome'] >= 3000) &
(filter_data['AverageIncome'] <= 5000)]
print("Квартиросъемщики с зарплатой от 3000 до 5000:")
print(income_range)

# 5. Получающие субсидию, чья квартплата превышает 1500
subsidy_high_rent = filter_data[(filter_data['Subsidy'] > 0) &
(filter_data['Rent'] > 1500)]
print("Получающие субсидию с квартплатой более 1500:")
print(subsidy_high_rent)

# 6. Получающие субсидию с квартплатой свыше 1800 или имеющие
зарплату менее 5000 и задолженность
subsidy_or_debt = filter_data[((filter_data['Subsidy'] > 0) &
(filter_data['Rent'] > 1800)) |
                                ((filter_data['AverageIncome'] <
5000) & (filter_data['Debts'] > 0))]
print("Получающие субсидию с квартплатой более 1800 или зарплата <
5000 с задолженностью:")
print(subsidy_or_debt)

# 7. Задолженность по квартплате, квартплата не превышает 1600 или
зарплата от 3000 до 5000 и квартплата более 1800
debt_rent_income = filter_data[((filter_data['Debts'] > 0) &
(filter_data['Rent'] <= 1600)) |
                                ((filter_data['AverageIncome'] >=
3000) & (filter_data['AverageIncome'] <= 5000) & (filter_data['Rent'] >
1800))]
print("Задолженность по квартплате, квартплата <= 1600 или зарплата
от 3000 до 5000 и квартплата > 1800:")
print(debt_rent_income)

```

Результат тестов:

```
Получающие субсидию:
  Adress      Tenant  AverageIncome  Debts  Rent  Subsidy
2   Gogol_3_4  Ulanova TI           3200    0.0  1200   120.0
7  Korolova_2_56  Ivanov II           2900    0.0    800    80.0
11 Korolova_2_60  Sidorov VK           4400    0.0   1900   190.0
Зарабатывающие меньше 4500 и имеющие задолженность:
Empty DataFrame
Columns: [Adress, Tenant, AverageIncome, Debts, Rent, Subsidy]
Index: []
Квартиросъемщики с квартплатой больше 2000:
  Adress      Tenant  AverageIncome  Debts  Rent  Subsidy
3   Gogol_3_5  Budanova AI           6700    0.0   2100    0.0
10  Korolova_2_59  Petrov MM          10000    0.0   2300    0.0
Квартиросъемщики с зарплатой от 3000 до 5000:
  Adress      Tenant  AverageIncome  Debts  Rent  Subsidy
0   Gogol_3_2  Ponomarev VD           5000    0.0   1500    0.0
1   Gogol_3_3  Ponomarev AD           4500    0.0   2000    0.0
2   Gogol_3_4  Ulanova TI           3200    0.0   1200   120.0
5   Gogol_3_7  Komarov DP           4900    0.0   1000    0.0
11  Korolova_2_60  Sidorov VK           4400    0.0   1900   190.0
Получающие субсидию с квартплатой более 1500:
  Adress      Tenant  AverageIncome  Debts  Rent  Subsidy
11  Korolova_2_60  Sidorov VK           4400    0.0   1900   190.0
Получающие субсидию с квартплатой более 1800 или зарплата < 5000 с задолженностью:
  Adress      Tenant  AverageIncome  Debts  Rent  Subsidy
11  Korolova_2_60  Sidorov VK           4400    0.0   1900   190.0
Задолженность по квартплате, квартплата <= 1600 или зарплата от 3000 до 5000 и квартплата > 1800:
  Adress      Tenant  AverageIncome  Debts  Rent  Subsidy
1   Gogol_3_3  Ponomarev AD           4500    0.0   2000    0.0
11  Korolova_2_60  Sidorov VK           4400    0.0   1900   190.0
```

Задание 3

Условие :

1. По данным из файлов students1.txt, students2.txt, students3.txt сделать вывод о линейной зависимости между успеваемостью студентов по следующим предметам, посчитав соответствующие коэффициенты корреляции оценок по

- 1) 2 семестрам программирования и алгоритмам (Програмування1, Програмування2, Алгоритми і структури даних) (попарно для 3 предметов)
- 2) культурологии и функциональному анализу (Історія української культури, Функціональний аналіз)

Код решения:

```
import pandas as pd
```

```

# Загрузка данных из текстовых файлов
students1 = pd.read_csv('students1.txt', sep='\t')
students2 = pd.read_csv('students2.txt', sep='\t')

# Объединение данных из двух таблиц по общему столбцу 'Name'
students_data = pd.merge(students1, students2, on='Name', how='inner')

# Расчет коэффициентов корреляции между заданными предметами
correlation_prog1_prog2 =
students_data['Програмування1'].corr(students_data['Програмування2'])
correlation_prog1_algorithms =
students_data['Програмування1'].corr(students_data['Алгоритми і
структури даних'])
correlation_prog2_algorithms =
students_data['Програмування2'].corr(students_data['Алгоритми і
структури даних'])

correlation_culture_analysis = students_data['Історія української
культури'].corr(students_data['Функціональний аналіз'])

# Вывод коэффициентов корреляции
print(f'Коеффіцієнт кореляції между Програмування1 и Програмування2:
{correlation_prog1_prog2:.2f}')
print(f'Коеффіцієнт кореляції между Програмування1 и Алгоритми і
структури даних: {correlation_prog1_algorithms:.2f}')
print(f'Коеффіцієнт кореляції между Програмування2 и Алгоритми і
структури даних: {correlation_prog2_algorithms:.2f}')
print(f'Коеффіцієнт кореляції между Історія української культури и
Функціональний аналіз: {correlation_culture_analysis:.2f}')

```

Результат тестов:

```

Коеффіцієнт кореляції между Програмування1 и Програмування2: 0.67
Коеффіцієнт кореляції между Програмування1 и Алгоритми і структури даних: 0.61
Коеффіцієнт кореляції между Програмування2 и Алгоритми і структури даних: 0.76
Коеффіцієнт кореляції между Історія української культури и Функціональний аналіз: 0.49

```

Задание 4

Условие :

1. По данным из файлов 2008.csv.bz2 (или 2008_rand.csv) и airports.csv выполнить следующие задания:

- 1) Найдите среднее, минимальное и максимальное время полёта.
- 2) Не выглядит ли подозрительным минимальное время полёта? В какие дни и на каких рейсах оно было? Какое время полёта было у этих же рейсов в другие дни?
- 3) Найти аэропорт, в который производилось наибольшее количество полётов.
- 4) Определить 3 наиболее популярных места назначения (Dest) в июне. В каких городах они находятся?
- 5) Найти аэропорт, в котором была наибольшая задержка при посадке по причине погоды (WeatherDelay), а также количество минут опоздания. В какой день, месяц и год это произошло?

Код решения:

```
import pandas as pd

# Загрузка данных из файлов
flights_data = pd.read_csv('2008_rand.csv')
airports_data = pd.read_csv('airports.csv')

# 1. Среднее, минимальное и максимальное время полёта
mean_flight_time = flights_data['AirTime'].mean()
min_flight_time = flights_data['AirTime'].min()
max_flight_time = flights_data['AirTime'].max()

print(f"Среднее время полета: {mean_flight_time}")
print(f"Минимальное время полета: {min_flight_time}")
print(f"Максимальное время полета: {max_flight_time}")

# 2. Анализ минимального времени полёта
suspicious_flights = flights_data[flights_data['AirTime'] ==
min_flight_time]
print("Рейсы с минимальным временем полета:")
print(suspicious_flights)

# 2. Время полета этих же рейсов в другие дни
```

```

same_flights = flights_data.merge(suspicious_flights[['FlightNum',
'Origin', 'Dest']], on=['FlightNum', 'Origin', 'Dest'])
same_flights = same_flights[same_flights['AirTime'] != min_flight_time]
print("Время полета тех же рейсов в другие дни:")
print(same_flights[['FlightNum', 'AirTime']])

# 3. Аэропорт с наибольшим количеством полетов
most_flights_airport = flights_data['Dest'].value_counts().idxmax()
most_flights_airport_name = airports_data[airports_data['iata'] ==
most_flights_airport]['airport'].values[0]
print(f"Аэропорт с наибольшим количеством полетов:
{most_flights_airport_name}")

# 4. Три наиболее популярных места назначения в июне
june_flights = flights_data[flights_data['Month'] == 6]
top_3_destinations = june_flights['Dest'].value_counts().head(3)
destinations_info =
airports_data[airports_data['iata'].isin(top_3_destinations.index)]

# Используем apply для вывода популярных мест без цикла for
print(destinations_info.apply(lambda row: f"Популярное место
назначения: {row['iata']}, город: {row['city']}", axis=1).tolist())

# 5. Аэропорт с наибольшей задержкой при посадке по причине погоды
max_weather_delay = flights_data['WeatherDelay'].max()
max_weather_delay_flight = flights_data[flights_data['WeatherDelay'] ==
max_weather_delay].iloc[0]
delay_airport = max_weather_delay_flight['Dest']
delay_airport_name = airports_data[airports_data['iata'] ==
delay_airport]['airport'].values[0]
print(f"Аэропорт с наибольшей задержкой по погоде:
{delay_airport_name}, задержка: {max_weather_delay} минут, дата:
{max_weather_delay_flight['Month']}/{max_weather_delay_flight['DayofMon
th']}/2008")

```

Результат тестов:

```

Среднее время полета: 103.8296288873164
Минимальное время полета: 0.0
Максимальное время полета: 629.0
Рейсы с минимальным временем полета:
  Unnamed: 0  Year  Month  DayofMonth  ...  WeatherDelay  NASDelay  SecurityDelay  LateAircraftDelay
377833      1374982  2008      3          18  ...          0.0        97.0          0.0          0.0

[1 rows x 30 columns]
Время полета тех же рейсов в другие дни:
  FlightNum  AirTime
0         5854    130.0
1         5854    139.0
2         5854    131.0
3         5854     NaN
5         5854    128.0
Аэропорт с наибольшим количеством полетов: William B Hartsfield-Atlanta Intl
['Популярное место назначения: ATL, город: Atlanta', 'Популярное место назначения: DFW, город: Dallas-Fort Worth', 'Популярно
е место назначения: ORD, город: Chicago']
Аэропорт с наибольшей задержкой по погоде: Detroit Metropolitan-Wayne County, задержка: 1297.0 минут, дата: 12/17/2008

```